

Constraint Satisfaction Problem (CSP) & Map Coloring using AC-3 Algorithm

Learning Objectives

- Understand CSP's components
 - Understand Map Coloring as a CSP
 - Apply Arc Consistency (AC-3)
 - Analyze domain reduction using constraint propagation
-

1. Introduction to CSP

A **Constraint Satisfaction Problem (CSP)** is a class of problems where the goal is not to find a path or optimize a cost, but to **find a valid configuration of values that satisfies a set of restrictions**. In other words, the problem is about feasibility rather than optimization—we are interested in whether a consistent assignment exists, and if so, what it is.

CSPs are especially useful in situations where multiple conditions must be satisfied simultaneously, and where making one decision directly affects the possibilities of others. Unlike traditional search or pathfinding algorithms (like BFS or A*), which focus on reaching a goal through a sequence of steps, **CSPs focus on assigning values in such a way that all constraints are respected at once.**

The real strength of CSPs lies in their ability to **systematically reduce the search space using constraints**. Techniques like constraint propagation (e.g., AC-3) eliminate impossible choices early, making them far more efficient than brute-force approaches for certain problems.

Some important categories of problems where CSPs are the most natural and effective approach include:

- **Scheduling problems** (e.g., exam timetabling, class scheduling): where time slots, rooms, and resources must be assigned without conflicts
- **Assignment problems** (e.g., assigning tasks to workers): where constraints like skill compatibility or workload limits must be satisfied

- **Puzzle problems** (N-Queens): where every placement must follow strict rules
- **Map coloring problems**: where adjacent regions must differ
- **Configuration problems** (e.g., product configuration, circuit design): where components must fit together under constraints

These problems are difficult to solve efficiently using standard algorithms like greedy methods or uninformed search because those approaches **do not inherently account for global constraints**. CSP techniques, on the other hand, are designed specifically to handle such dependencies.

1.1 Components of CSP:

A Constraint Satisfaction Problem is defined by three components:

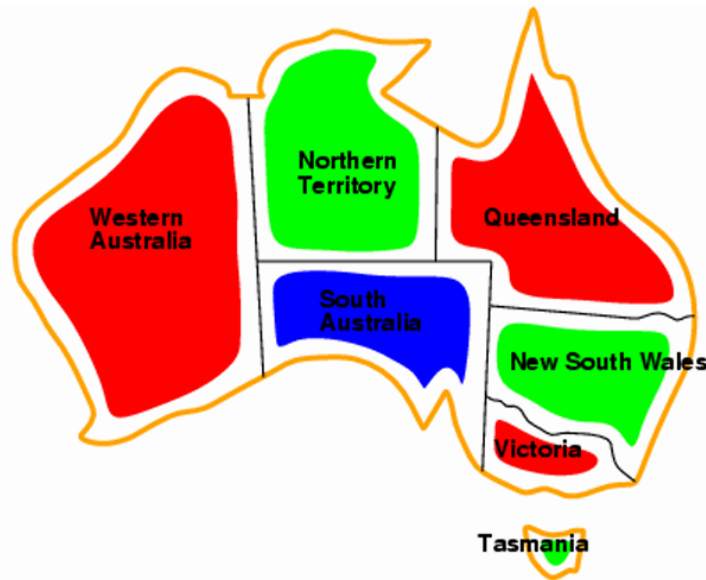
- **Variables (X)**: A finite set of unknowns, e.g. {S, E, N, D, M, O, R, Y}.
- **Domains (D)**: A set of possible values for each variable, e.g. {0, 1, 2, ..., 9}.
- **Constraints (C)**: Rules that restrict which combinations of values are legal, e.g. in (S,E,N,D,M,O,R,Y) and $S \neq 0$, $M \neq 0$.

The goal is to find an assignment of values to variables such that every constraint is satisfied. CSP techniques are used in scheduling, resource allocation, puzzle solving, and many other AI domains

2. Map Coloring Example



Colored Map Solution:



3. CSP Formulation

Variables: WA, NT, Q, NSW, V, SA, T

Domains: {Red, Green, Blue}

Constraints:

- $WA \neq NT$
- $WA \neq SA$
- $NT \neq SA$
- $NT \neq Q$
- $SA \neq Q$
- $SA \neq NSW$
- $SA \neq V$
- $Q \neq NSW$
- $NSW \neq V$

Note: Tasmania (T) has no neighbors.

4. Arc Consistency (AC-3)

Arc consistency is a technique used in CSPs to reduce the number of possible values (domains) of variables by checking consistency between pairs of variables. It ensures that **for every value of one variable, there is at least one compatible value in its neighboring variable that satisfies the constraint between them.**

If a value of a variable does not have any supporting value in its neighbor, it is considered invalid and removed from the domain. This process helps in eliminating impossible choices early, before actually trying to assign final values.

The idea becomes powerful when applied repeatedly across all variable pairs. When we enforce arc consistency throughout the entire problem, we gradually shrink the domains of variables, making the problem simpler and easier to solve.

This is exactly what the AC-3 algorithm does. It systematically checks all arcs (pairs of variables), removes inconsistent values, and propagates these changes to other related variables. By doing so, **AC-3 reduces the search space significantly and can even detect failure early** if any variable is left with an empty domain.

However, it is important to note that arc consistency alone does not always guarantee a complete solution, but it makes the remaining problem much easier to solve using search techniques like backtracking.

5. AC-3 Pseudocode

function AC-3(csp):

queue $\leftarrow$ all arcs (X_i, X_j)

while queue is not empty:

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$

if REVISE(X_i, X_j):

if domain of X_i is empty:

return FAILURE

for each X_k in neighbors(X_i) where $X_k \neq X_j$:

add (X_k, X_i) to queue

return SUCCESS

function REVISE(X_i, X_j):

revised $\leftarrow$ false

for each value x in domain(X_i):

if no value y in domain(X_j) satisfies constraint:

remove x from domain(X_i)

revised $\leftarrow$ true

return revised

Lab Tasks

Task 1: Write a Python program to implement the AC-3 algorithm for the given Australia map coloring problem. Your program should:

- Define variables, domains, and constraints
- Implement the **REVISE function**
- Implement the **AC-3 algorithm**
- Display **domain values after each revision step**
- Print the **final reduced domains**

Focus: Show how domains shrink step-by-step ... not just final output

Task 2: Extend your AC-3 program to assign final colors to all regions of map using **Backtracking Search** after applying AC-3. Your program should:

- Take reduced domains from AC-3
- Assign values such that all constraints are satisfied
- Print the **final valid coloring of the map**

Lab # 13 Marks distribution

	ER1	ER6	ER8
Task	3 points	3 points	4 points

Lab # 13 Rubric Evaluation Guideline:

#	Qualities & Criteria	0 < Poor <= 1	1 < Satisfactory <= 2	2 < Excellent <=3
ER1	Task Completion	Minimal or no program functionality was achieved.	Some tasks were completed, but the program has errors or incomplete functionalities.	All tasks were completed, and the program runs without errors.
#	Qualities & Criteria	0 < Poor <= 1	1 < Satisfactory <= 2	2 < Excellent <=3
ER6	Program Output	Output is inaccurate or poorly presented.	Output is mostly accurate but may lack labels, captions, or formatting.	Output is clear, accurate, and well presented with labels, captions, and proper formatting.
#	Qualities & Criteria	0 < Poor <= 1	1.5 < Satisfactory <= 2.5	2.5 < Excellent <= 4
ER8	Question & Answer	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.